UT1 · KOTLIN FUNDAMENTALS

Block 2

Conditionals, ranges and loops

What you will learn	Session
Make decisions with `if`, `else if` and `else`	3
Use `if` as an expression to store the result of a condition	3
Replace long `if` chains with `when`, with and without a subject	3
Create ranges with `..`, `..<`, `downTo` and `step`, and check values with `in`	3
Repeat code with `for`, `while` and `do-while`	4
Control loops with `break` and `continue`	4

In Block 1 every program ran from top to bottom, line by line. Real programs need to **decide** what to do (conditionals) and **repeat** work (loops). You already know these ideas from Java; in Kotlin they look similar, but two of them are much more powerful: `if` and `when` can **return a value**. You will use this constantly in Jetpack Compose, for example to decide which text or colour to show on the screen.

Contents

Contenido

Contents....1
1. Conditionals: if, else if and else....3
2. if as an expression....4
3. The when expression....5
3.1 when with a subject....5
3.2 when as an expression....6
3.3 when without a subject....6
4. Ranges....7
4.1 Checking a value with in and !in....8
4.2 Ranges inside when....8
5. The for loop....8
5.1 Accumulating results....9

6. while and do-while 10
6.1 do-while 10
7. break and continue 11
8. Which one should I use? 11
9. Summary: Java vs Kotlin 12
10. Exercises 13
Exercise 1 · Shipping cost 13
Exercise 2 · From mark to grade 13
Exercise 3 · Multiplication table 13
Exercise 4 · Predict the output 13
Exercise 5 · Month of the year 14
Exercise 6 · Month and season 14
Challenge · Number guessing game 15

1. Conditionals: if, else if and else

An `if` runs a block of code only when a condition is `true`. The condition must always be a `Boolean`: Kotlin does not accept numbers or text as conditions.

```
fun main() {
    val age = 17

    if (age >= 18) {
        println("You can vote")
    } else {
        println("You cannot vote yet")
    }
}
```

When there are more than two possibilities, chain conditions with `else if`. Kotlin checks them **from top to bottom** and runs only the **first** block whose condition is true; the rest are skipped.

```
fun main() {
    val orderTotal = 65.0

    if (orderTotal >= 100) {
        println("Free shipping")
    } else if (orderTotal >= 50) {
        println("Shipping with discount")
    } else {
        println("Standard shipping cost")
    }
}
```

Output

```
Shipping with discount
```

Conditions can be combined with the logical operators from Block 1 (`&&`, `||`, `!`):

```
fun main() {
    val price = 175.0
    val isPremiumCustomer = false

    if (price >= 100 || isPremiumCustomer) {
        println("Free shipping")
    }
}
```

TIP · Always use braces

If a block has a single line, the braces `{ }` are optional: `if (age >= 18) println("Adult")`. Even so, writing them is a good habit: when you later add a second line to the block, forgetting the braces creates a bug that is very hard to see.

2. if as an expression

In Java, `if` is a **statement**: it does something, but it does not produce a value. In Kotlin, `if` is also an **expression**: it produces a value that you can store in a variable, pass to a function or print directly.

```
fun main() {
    val age = 33

    val accessMessage = if (age >= 18) "Adult" else "Under-age"

    println(accessMessage)   // Adult
}
```

Compare it with the version without expressions. Both work, but the expression version is shorter, avoids a `var`, and makes it impossible to forget to assign a value:

```
fun main() {
    val age = 33

    // Without if as an expression: needs a var
    var message1 = ""
    if (age >= 18) {
        message1 = "Adult"
    } else {
        message1 = "Under-age"
    }

    // With if as an expression: a single val
    val message2 = if (age >= 18) "Adult" else "Under-age"
    println("$message1 $message2")
}
```

When the branches contain blocks with several lines, the value of each branch is its **last expression**:

```
fun main() {
    val orderTotal = 65.0

    val shippingCost = if (orderTotal >= 100) {
        println("Congratulations, shipping is free!")
        0.0
    } else {
        println("Spend ${100 - orderTotal} more for free shipping")
        4.99
    }

    println("Shipping: $shippingCost")
}
```

WATCH OUT · else is mandatory in an expression

When you use `if` as an expression, the `else` branch is **required**: the variable must receive a value in every possible case. Without it, the code does not compile.

> **COMING FROM JAVA · There is no ternary operator**
> Kotlin does not have Java's `condition ? a : b`. The `if` expression replaces it:
> Java: `String msg = age >= 18 ? "Adult" : "Under-age";`
> Kotlin: `val msg = if (age >= 18) "Adult" else "Under-age"`

You can even use an `if` expression inside a string template, which is very common in Compose:

```
fun main() {
    val unreadMessages = 1
    println("You have $unreadMessages ${if (unreadMessages == 1) "message" else
"messages"}")
}
```

3. The when expression

`when` chooses between several options in a clean, readable way. It replaces long `if / else if` chains and Java's `switch`.

3.1 when with a subject

Write the value to check between parentheses (the **subject**). Each branch has the form `value -> action`. Kotlin compares the subject with each value from top to bottom and runs **only the first branch that matches**. The `else` branch runs when nothing else matches.

```
fun main() {
    val userRole = "editor"

    when (userRole) {
        "admin" -> println("Hello Admin")
        "editor" -> println("Hello Editor")
        "viewer" -> println("Hello Viewer")
        else -> println("Unknown role")
    }
}
```

A branch can accept several values separated by commas, and can run several lines if you use braces:

```
fun main() {
    val day = 6

    when (day) {
        1, 2, 3, 4, 5 -> println("Weekday")
        6, 7 -> {
            println("Weekend")
            println("No classes today!")
        }
        else -> println("Invalid day")
    }
}
```

COMING FROM JAVA · when vs switch

There is no `case` and no `break`. Each branch ends automatically, so the "fall-through" bug of Java's `switch` (forgetting a `break` and running the next case too) cannot happen in Kotlin.

`when` works with any type: numbers, `String`, `Char`, `Boolean`… and also with ranges, as you will see in section 4.

3.2 when as an expression

Like `if`, `when` can return a value. In that case the `else` branch is **mandatory** (unless the compiler can prove that every possible value is covered, which happens with some special types we will see later).

```
fun main() {
    val userRole = "viewer"

    val permission = when (userRole) {
        "admin" -> "Full access"
        "editor" -> "Editing access"
        "viewer" -> "Read-only access"
        else -> "No access"
    }

    println(permission)   // Read-only access
}
```

3.3 when without a subject

If you leave out the subject, each branch is a complete `Boolean` condition. This is the cleanest way to write an `if / else if` chain:

```
fun main() {
    val temperature = 31

    val advice = when {
        temperature < 0 -> "Danger: ice on the road"
        temperature < 15 -> "Take a coat"
        temperature < 30 -> "Nice weather"
        else -> "Stay in the shade"
    }

    println(advice)   // Stay in the shade
}
```

WATCH OUT · The order of the branches matters

Only the **first** matching branch runs. Put the most specific conditions first. In this example the second branch can never run, because any score greater than 90 is also greater than 50 and the first branch catches it:

```
score > 50 -> "Passed"
score > 90 -> "Excellent"   // never reached!
```

Swap them, or write the conditions so that they do not overlap.

4. Ranges

A **range** represents all the values between a start and an end. Ranges are used to check whether a value is inside an interval and to control `for` loops.

Syntax	Values	Meaning
`1..5`	1, 2, 3, 4, 5	Closed range: the end **is** included
`1..<5`	1, 2, 3, 4	Open-ended range: the end is **not** included
`1 until 5`	1, 2, 3, 4	Same as `1..<5` (older syntax)
`5 downTo 1`	5, 4, 3, 2, 1	Descending range
`1..10 step 2`	1, 3, 5, 7, 9	Jumps of 2
`10 downTo 0 step 5`	10, 5, 0	Descending with jumps
`'a'..'e'`	a, b, c, d, e	Ranges also work with characters

WATCH OUT · 5..1 is empty

`..` always counts **upwards**. A range like `5..1` does not count down: it is simply empty, so a loop over it runs zero times and no error is shown. To count down, use `downTo`.

4.1 Checking a value with in and !in

The operator `in` checks whether a value is inside a range, and `!in` checks that it is outside. They replace the double comparisons you would write in Java:

Java	Kotlin

`if (level >= 1 && level <= 10) {`
`  // valid level`
`}`
	`if (level in 1..10) {`
	`  // valid level`
	`}`

```
fun main() {
    val number = 15

    if (number !in 1..10) {
        println("Please enter a number between 1 and 10")
    }

    val letter = 'k'
    println(letter in 'a'..'z')   // true
}
```

4.2 Ranges inside when

`in` can be used directly in the branches of a `when`. This is perfect for classifying values into intervals:

```
fun main() {
    val mark = 7

    val grade = when (mark) {
        in 0..4 -> "Fail"
        5 -> "Pass"
        6 -> "Good"
        in 7..8 -> "Very good"
        in 9..10 -> "Excellent"
        else -> "Invalid mark"
    }

    println(grade)   // Very good
}
```

TIP · Avoid overlapping ranges

If two ranges share a value, for example `in 0..3` and `in 3..5`, the value 3 always goes to the first branch. The code compiles, but it is usually a mistake. Make each range start right after the previous one ends.

5. The for loop

A `for` loop repeats a block once for **each element** of something that can be traversed: a range, a text, and, later, a list. Its form is always `for (variable in collection)`.

```
fun main() {
    for (lessonNumber in 1..5) {
        println("Class number $lessonNumber")
    }
}
```

Output

```
Class number 1
Class number 2
Class number 3
Class number 4
Class number 5
```

Any range from section 4 can be used in a `for`:

```
fun main() {
    for (i in 0..<3) print("$i ")          // 0 1 2
    println()
    for (i in 10 downTo 1) print("$i ")    // 10 9 8 ... 1
    println()
    for (i in 0..20 step 5) print("$i ")   // 0 5 10 15 20
    println()
    for (letter in "Kotlin") print("$letter-")   // K-o-t-l-i-n-
    println()
}
```

COMING FROM JAVA · There is no classic for(;;)

Kotlin does not have Java's `for (int i = 0; i < 5; i++)`. Use a range instead: `for (i in 0..<5)`.

The loop variable is read-only (like a `val`) and you do not declare its type: `for (i in 1..5)`, not `for (var i: Int in 1..5)`.

5.1 Accumulating results

A very common pattern is to keep a result in a `var` declared **before** the loop and update it on each iteration. Pay attention to where you declare that variable:

```
fun main() {
    var total = 0                 // declared OUTSIDE: keeps its value
    for (n in 1..5) {
        total += n
    }
    println("Sum 1..5 = $total")  // 15

    for (n in 1..3) {
        var counter = 0           // declared INSIDE: reset on every iteration
        counter++
        println(counter)          // always prints 1
    }
}
```

6. while and do-while

A `while` loop repeats a block **as long as** a condition is true. Use it when you do not know in advance how many times the loop will run: waiting for a valid input, repeating until a game ends, etc.

```
fun main() {
    var batteryLevel = 100

    while (batteryLevel > 0) {
        println("Battery: $batteryLevel%")
        batteryLevel -= 25
    }
    println("Battery empty")
}
```

Output

```
Battery: 100%
Battery: 75%
Battery: 50%
Battery: 25%
Battery empty
```

WATCH OUT · Infinite loops

Something inside the loop must eventually make the condition `false`. If you forget `batteryLevel -= 25`, the loop never ends and the program hangs. In IntelliJ, stop it with the red ■ button in the Run window.

6.1 do-while

`do-while` checks the condition **at the end**, so the block always runs **at least once**. That makes it ideal for asking the user for data until it is valid:

```
fun main() {
    var number: Int
    do {
        print("Enter a number between 1 and 10: ")
        number = readln().toInt()
    } while (number !in 1..10)

    println("Thank you, you entered $number")
}
```

Example run

```
Enter a number between 1 and 10: 25
Enter a number between 1 and 10: 0
Enter a number between 1 and 10: 7
Thank you, you entered 7
```

With a normal `while`, the condition is checked first, so the block may run zero times. With `do-while` it always runs at least once.

7. break and continue

Two keywords change the normal flow of a loop:

- `break` **exits the loop** immediately. The program continues after the loop.
- `continue` **skips the rest of the current iteration** and jumps to the next one.

```
fun main() {
    // break: stop searching when we find the number
    for (number in 1..10) {
        if (number == 4) {
            println("Found it!")
            break
        }
        println(number)
    }
    // prints 1 2 3 Found it!

    // continue: skip the odd numbers
    for (number in 1..10) {
        if (number % 2 != 0) {
            continue
        }
        print("$number ")
    }
    // prints 2 4 6 8 10
}
```

8. Which one should I use?

Situation	Best choice
Two options (yes/no)	`if / else`
Three or more options based on one value	`when (value)`
Several unrelated conditions	`when { }` without subject
Classifying a number into intervals	`when` with `in` ranges
Storing the result of a decision	`val x = if …` or `val x = when …`
Repeating a known number of times	`for` with a range
Traversing the characters of a text	`for (c in text)`
Repeating until something happens	`while`

Situation	Best choice
Repeating at least once (e.g. input validation)	`do-while`

9. Summary: Java vs Kotlin

Concept	Java	Kotlin
Conditional value	`a > b ? x : y`	`if (a > b) x else y`
Multiple choice	`switch` / `case` / `break`	`when` with `->`
Several values per case	`case 6: case 7:`	`6, 7 ->`
Default case	`default:`	`else ->`
Value in interval	`x >= 1 && x <= 10`	`x in 1..10`
Counting loop	`for (int i = 0; i < 5; i++)`	`for (i in 0..<5)`
Counting down	`for (int i = 5; i >= 1; i--)`	`for (i in 5 downTo 1)`
while, do-while, break, continue	Same	Same

10. Exercises

Create one Kotlin file per exercise inside `UT1_KotlinBasics/src/exercises`. Use `val` unless you need `var`, prefer `if` and `when` as **expressions** when you are storing a result, and avoid magic numbers: store fixed values in constants.

Exercise 1 · Shipping cost

Ask the user for the total of an order. Using a single `val` and `if` as an expression, calculate the shipping cost: free if the total is 50 € or more, 4.99 € otherwise. Print the total, the shipping cost and the final amount with two decimals.

> Example
```
Order total: 42.5
Shipping: 4.99
To pay: 47.49
```

Exercise 2 · From mark to grade

Ask the user for a mark from 0 to 10 (`Int`) and show the grade using `when` with ranges: 0–4 "Fail", 5 "Pass", 6 "Good", 7–8 "Very good", 9–10 "Excellent". Any other number must show "Invalid mark".

Exercise 3 · Multiplication table

Ask the user for a number and print its multiplication table from 1 to 10 using a `for` loop. Then print it again in reverse order (from 10 to 1).

> Example
```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Exercise 4 · Predict the output

Without running the code, write down what each fragment prints. Then run it and check your answers. Explain any difference.

```
fun main() {
    // a)
    for (i in 1..10 step 3) print("$i ")
    println()

    // b)
    for (i in 5..1) print("$i ")
    println("end")

    // c)
    val x = 12
    val r = when {
        x > 5 -> "big"
        x > 10 -> "huge"
        else -> "small"
    }
    println(r)

    // d)
    var n = 0
    do {
        n++
    } while (n < 0)
    println(n)

    // e)
    for (i in 1..6) {
        if (i == 3) continue
        if (i == 5) break
        print("$i ")
    }
    println()
}
```

Exercise 5 · Month of the year

Write a program that asks the user to enter a number from 1 to 12 and converts it into the name of the corresponding month.

- If the user enters 1, the program must display "January".
- If the user enters 5, the program must display "May".
- If the user enters 12, the program must display "December".
- If the user enters a number that is not between 1 and 12, it must display "Invalid month".

Concepts you must use: `readln()`, `toInt()`, `when`, `else`.

Exercise 6 · Month and season

Extend the previous exercise. The program must display the name of the month **and** the season it belongs to:

Months	Season
12, 1, 2	Winter

Months	Season
3, 4, 5	Spring
6, 7, 8	Summer
9, 10, 11	Autumn

For example, 1 displays "January" and "Winter", and 10 displays "October" and "Autumn". A number outside 1–12 must display "Invalid month".

Concepts you must use: `readln()`, `toInt()`, `if`, ranges with `in`, `when`, `else`.

> **TIP · Think about it**
> Winter is the only season whose months are not consecutive. Why can you not write it as a single range? Find two different ways to solve it.

Challenge · Number guessing game

Write a program in which the user has to guess a secret number.

- The secret number is fixed (for example, 7), and the valid range is 1..10.
- The user has a maximum of 3 attempts.
- If the user enters a number outside the range, show an error message. That number does **not** count as an attempt.
- If the user guesses correctly, show a victory message and end the game.
- If the user guesses wrong, indicate whether the secret number is higher or lower.
- If the user runs out of attempts, show a defeat message.

The program must keep track of how many attempts have been used, how many are left, and whether the user has won or lost.

Concepts you must use: `val`, `var`, `readln()`, `toInt()`, `if / else if / else`, ranges with `in` or `!in`, `while`, `break`, `continue`.

Example run

```
Guess the number (1-10). You have 3 attempts.
Attempt 1: 15
15 is not between 1 and 10. Try again.
Attempt 1: 4
Wrong! The secret number is higher. Attempts left: 2
Attempt 2: 8
Wrong! The secret number is lower. Attempts left: 1
Attempt 3: 7
Congratulations, you guessed it in 3 attempts!
```